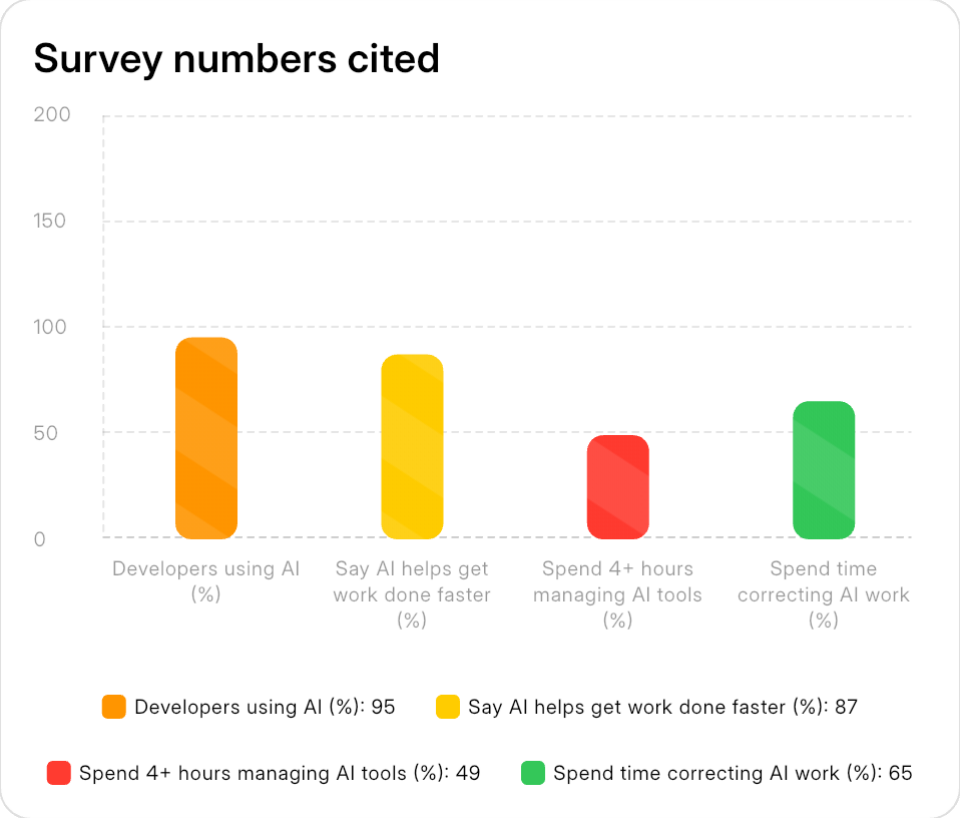
Summary

Human-AI Collaboration in Software Development

AI tools have made coding faster, but overall productivity is stuck at a 10–15% gain because coding is only 16–20% of a developer's time; the real wins come from redesigning workflows around humans and AI together—independent review agents, customer-insight pipelines, instant expert agents that cut wait time, and deliberately managed context for agents.

Key takeaways

- **The software factory / dark factory idea is rejected.** Software isn't built like cars on a production line; it takes back-and-forth communication and is built for humans, so humans stay in the loop.
- **AI adoption is near-total.** In their developer experience survey, 95% of developers use AI—the biggest adoption rate the speaker has seen in the industry.
- **Perceived speed hides real overhead.** 87% say AI helps them get work done faster, but 49% spend more than four hours managing AI tools and 65% spend time correcting AI's work.
- **Productivity is stuck at 10–15%** and has been for one to two years despite better tools and models. One explanation: coding is only ~16–20% of a developer's time, so AI only touches that slice.
- **More code creates a review bottleneck.** Initial review wait went from 32 to 71 minutes, and pull request size more than doubled from 46 to 99 lines of code.
- **"Looks good to me" risk.** AI-assisted reviews invite rubber-stamping, so changes need risk-based review according to their "blast radius".



Review pipeline for AI-written code

- A risk review agent assesses each change's blast radius; small changes (e.g., a button color) are auto-approved, while changes in critical areas like the payment system get human reviewers.
- Rule: two approvals and the change merges and builds automatically into staging, so no one waits for every reviewer.
- Instead of one generic PR review agent, they use several specialist agents—feature completeness, architecture, privacy, security—because a single reviewer, like a single interviewer, gives only one opinion.
- Agents are selected per review rather than always running all of them, to control token cost.
- The pipeline is still evolving: "the playbook hasn't been written yet".
- They are experimenting with AI-generated videos that explain a pull request's changes and architecture, to replace another wall of text.

Pull request review flow



Customer insight pipeline

- Customer interviews run over Zoom/Teams; an AI notetaker produces the transcript, and a Customer Insight Agent fills a pre-existing interview framework (communication, observation, problems, opportunities).
- The product manager who attended the interview must review the generated page to prevent AI hallucination or interpretation from steering what gets built.
- Pages are explicitly labeled "AI generated"—acceptance rates turned out higher than when output is presented as human-written.
- The full recording is attached to the page, and a Slack summary goes to all other product managers.
- Output rose from one customer interview per week to five per week.
- Because nobody reads a growing pile of AI-generated pages, an Insight Theme Analyzer agent scans them weekly and builds a whiteboard, bubbling up themes raised by several customers.
- Product managers then meet weekly to discuss and decide on a human basis; AI surfaces topics, humans decide.
- Reported results: more focused discussions, features built on repeated customer signals rather than one loud request, and higher customer satisfaction in their data.

Workflow changes and expert agents

- Weekly demos were replaced by short recorded feature videos with comments and reactions on the timeline; engagement rose 50% versus the live demo calls.
- Teams still meet for architectural discussions; the videos serve feature discussion.
- The number one developer productivity problem is wait time—handoffs and waiting on other departments.
- Agent "experts" now remove some of that wait:
  - A QA expert agent, built from QA engineers' instructions plus a Confluence knowledge base, lets developers run test plans themselves.
  - A release-note agent handles about 60% of cases well; larger release notes still need a technical writer or a technical writer's review.
  - A legal agent answers open-source license questions without raising a ticket.
- Roles are blending: designers prototype in code, product managers design with a design agent that knows the design language—but responsibility stays with the experts (developers for software, designers for UX, product managers for building the right thing).
- These agents are easy to build and always available, but they are not a replacement for human expertise and review.

Context is the real advantage

- The goal isn't just faster delivery; it is building the right thing, validating what matters, reacting immediately, and moving more independently.
- Agents now write code too, so the agent experience matters; models are the engine, but context is the fuel, and competitors use the same models—your context is the differentiator.
- Context lives everywhere—Jira items, chat messages, comments, meeting recordings, trade-off documents, the people who can validate—not just in the latest spec.
- Five failure patterns when agents lose context:
    - Specification ambiguity: a good agent asks, a bad one wanders.
    - The weekend runner: a vague goal ("go to the horizon") burns tokens until Monday.
    - Confident hallucination: the agent invents how to get from A to B.
    - Lost in the middle: too much context, mixing old and new data.
    - The zero-click leap: the agent acts on things it found (e.g., database credentials) without being asked.
- The CAFES framework (from some of the inventors of the Jobs-to-be-Done framework) covers clarity, efficiency and security among the five factors for writing context.
- Their working method: AI drafts the plan, humans edit it to add their own context and discuss it, then AI challenges the final plan.
- The old shape of work (light plan, execution, quick validation because you wrote it) is inverted: planning and validation now need the effort, while execution is effortless.

Speaker insights

- The speaker frames their own change of identity: from producer of software to managing agents and the software development lifecycle—while still collaborating with human colleagues.
- Changing how we work is the hardest part of the AI transformation, requiring whole new workflows rather than just using an agent to write code.
- No one knows how this ends; the frameworks haven't been written.
- The closing message: AI won't tell us what to do—imagination, experience and vision keep humans in the driver's seat, and the goal of building great software is unchanged.

Audience engagement

- The speaker opened by asking whether any agents were in the room and joked that some might be listening.
- On the invented user story "As a developer, I want the AI to take over coding so I have more time writing the specs," the speaker asked whether anyone had heard it from Anthropic; no one responded, which the speaker took as proof no developer would ever say it.

- Asking who likes reading AI-generated pages drew a telling audience reaction used as evidence for the theme-analyzer whiteboard.